

Python: Fundamentals

Cheng-Te Li (李政德)

`chengte@mail.ncku.edu.tw`

Dept. of Statistics, NCKU



Python程式編輯器
寫完整程式用

命令提示字元(終端機)
執行程式用

Python互動式直譯器
測試程式敘述用

The image displays three side-by-side screenshots illustrating the Python development workflow:

- Left Screenshot (Sublime Text):** Shows a Python file named `hello.py` with the code `print("Hello World")` on line 1. A yellow box highlights the line number '1'. Below the code, the text '第幾行' (Which line) is written in yellow.
- Middle Screenshot (Command Prompt):** Shows the execution of the Python script. The command `cd c:\Python35-32\workspace` is entered, followed by `>python hello.py`. A green box highlights the command `>python hello.py`, and the text '執行python程式' (Execute Python program) is written in green below it. The output 'Hello World' is displayed.
- Right Screenshot (Python Interactive Interpreter):** Shows the Python interactive shell. The command `>python` is entered, and a red box highlights it with the text '進入直譯器' (Enter interpreter) written in red. Below, several arithmetic expressions are entered and evaluated: `>>> 1+2` (output: 3), `>>> 3-5` (output: -2), `>>> 99*123` (output: 12177), and `>>> 12/5` (output: 2.4). Each input line is highlighted with an orange box.

Literal Constants (字面常數)

- **Fixed values** such as numbers, letters, and strings are called “**constants**” because their value does not change
- **Numeric constants** are as you expect
- **String constants** use **single quotes** (`'`) or **double quotes** (`"`)

```
123
```

```
99.909
```

```
"hello"
```

```
'good morning'
```

Numbers

- Integer number (整數)
 - 5, 100
- Floating point number (浮點數)
 - 3.14, -2.1999, 1.87e4
- Operators (運算子)

Operator	Description	Example	Result
+	addition	5 + 8	13
-	subtraction	90 - 10	80
*	multiplication	4 * 7	28
/	floating point division	7 / 2	3.5
//	integer (truncating) division	7 // 2	3
%	modulus (remainder)	7 % 3	1
**	exponentiation	3 ** 4	81

Numbers and Expression (運算式)

5

-99

05

1+2

1+2+3

3*5

2*-3*2

2*(-3)*2

3**5

```
>>> 05
      File "<stdin>", line 1
        05
         ^
      SyntaxError: invalid token
```

Expressions

Numbers

`/` carries out floating-point (decimal) division

`//` performs integer (truncating) division

`9/5`

`9//5`

`5/0`

`7//0`

`9%5`

`divmod(9,5)`

```
>>> 5 / 0
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
ZeroDivisionError: division by zero
```

```
>>> 7 // 0
```

```
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
```

```
ZeroDivisionError: integer division or modulo by zero
```

`10/5`

`10//5`

Mixing Integer and Floating

- When you perform an operation (運算子) where one operand (運算元) is an integer and the other operand is a floating point, the result is a floating point
- The integer is converted to a floating point before the operation

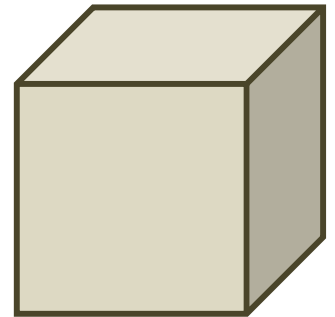
```
1 + 2.0
```

```
13 // 2 + 2
```

```
1 + 2 * 3 / 4.0 - 5
```

Object (物件)

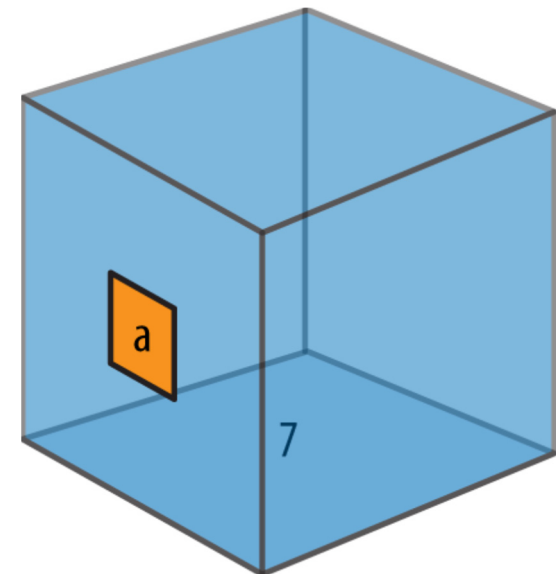
- Everything in Python is an **object**
 - integer, float, boolean, string, list, dictionary, function
- An object is like a **box** or a **container**
- The object has a **type** and a **value**
 - **7**: **integer** (**int**) 只能裝蘋果的箱子, 裝了7個蘋果
 - **9.3**: **float** 只能裝水的箱子, 裝了9.3公斤
 - **"hello"**: **string** (**str**) 只能裝文字的箱子, 裝了"hello"
- The **value** of an object can be
 - Changed (**mutable**)
 - Constant (**immutable**)



Variable

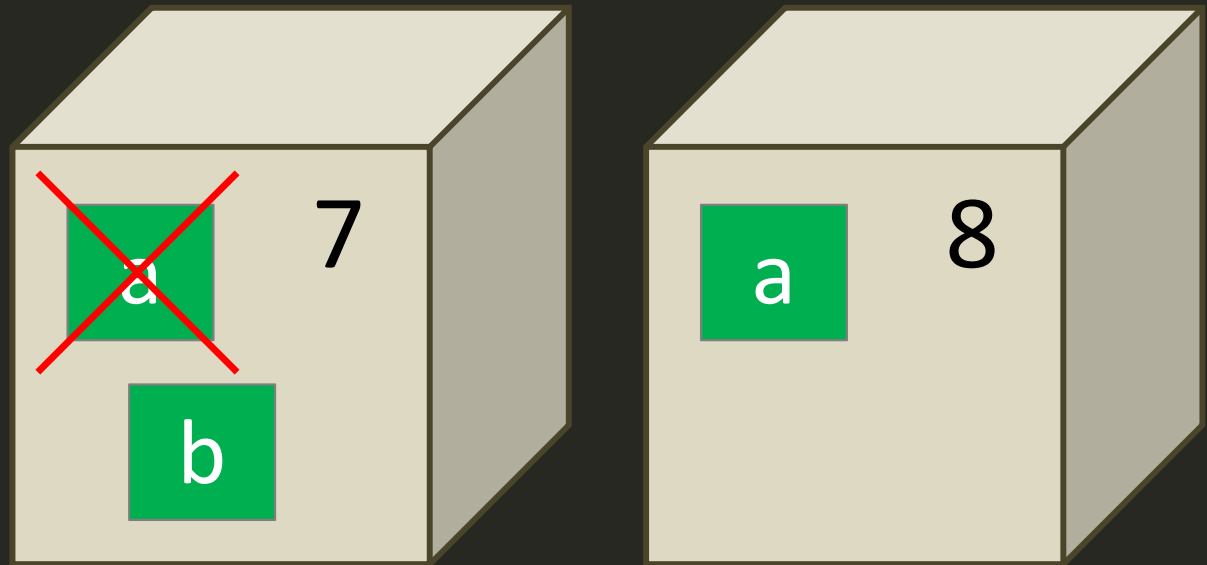
- You can define **variable**
 - A variable refers to a value you defined in the computer's memory
 - User **=** to **assign** a value to a variable
- Variable can be treated as a **tag** to an object

```
a = 7  
print(a)
```



Variable

```
a = 7  
print(a)  
b = a  
print(b)  
a = 8  
print(b)
```



Assignment Statements (敘述句)

- We assign a value to a variable using the **assignment statement (=)**
- An assignment statement consists of an **expression on the right-hand side** and a **variable** to store the result

$x = 4 * 6 - 7 + 12$

variable expression

Simultaneous Assignment

- An alternative form of the assignment statement is to calculate several values all at the same time

`<var>, <var>, ..., <var> = <expr>, <expr>, ..., <expr>`

```
a = 7
```

```
b = 8
```

```
a, b = 7, 8
```

```
m, n = a + b, a - b
```

```
x, y, z = a + b, a - b * 2 + 10, b**a
```

```
a, b = b, a
```

```
print(a, b)
```

Variable

- How to know the type of a variable/object?
 - Use `type(variable)` or `type(object)`

```
>>> a = 7
>>> b = a
>>> type(a)
<class 'int'>
>>> type(b)
<class 'int'>
>>> type(58)
<class 'int'>
>>> type(99.9)
<class 'float'>
>>> type('abc')
<class 'str'>
```

Variables

- What can be used to name the variables?
 - Lowercase letters (a through z)
 - Uppercase letters (A through Z)
 - Digits (0 through 9) Variables **cannot** begin with **digits**
 - Underscore (_) Variables can begin with **underscore**

```
a = 7
```

Valid

```
a1 = 7
```

```
a_b_c__99 = 7
```

```
_ab = 7
```

```
_1a = 7
```

```
(s = 7
```

Invalid

```
happ.y = 7
```

```
^_^ = 7
```

```
1_ = 7
```

```
1a = 7
```

```
^_^ = 7
```

SyntaxError: invalid syntax

Principle of Variable Naming

1. Must start with a letter or underscore _
 2. Must consist of letters and numbers and underscores
 3. **Case Sensitive** (有大小寫之別)
 4. Let the variable have some meaning
- **Good**: var, apple, _speed, temperature
 - **Different**: apple, Apple, APPLE

Mnemonic Variable Names

(=memory aid, 輔助記憶的)

```
x1q3_9ocd = 35.0  
x1q3_9XD = 12.50  
x1q3_9XD = x1q3_9ocd * x1q3_9XD  
print(x1q3_9afd)
```

What is this bit of code doing?

```
a = 35.0  
b = 12.50  
c = a * b  
print(c)
```

```
hours = 35.0  
rate = 12.50  
pay = hours * rate  
print(pay)
```

Variables Cannot be Reserved Words

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

These words, and some punctuation

. , : [] { } () # + - * / \ & ^ % \$ " ' < > ; | = ~

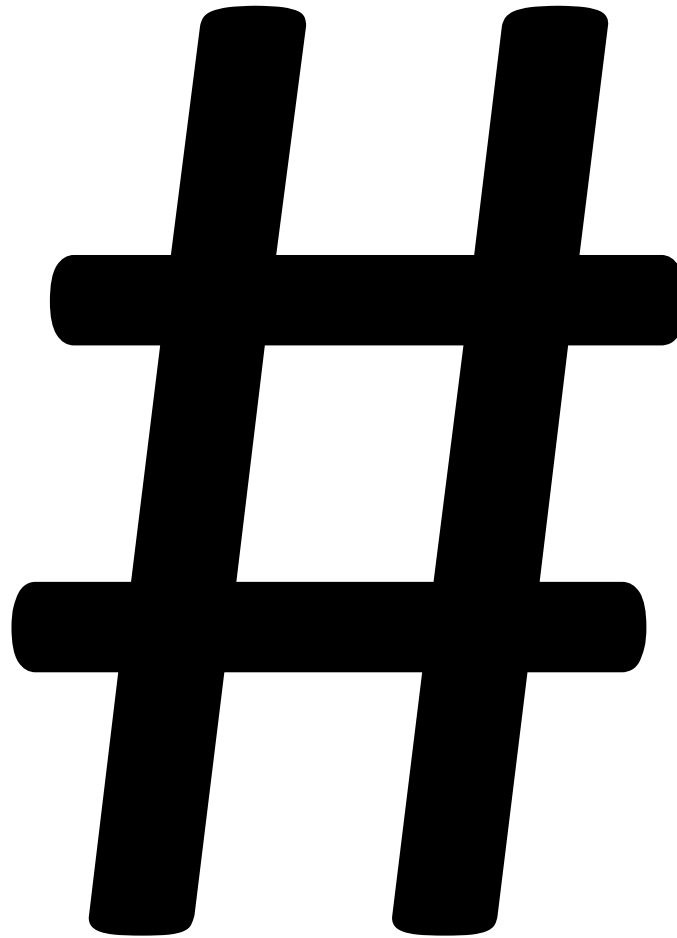
are used to define Python's syntax!

Hashtag

FB, IG, Twitter 的標籤

Number Sign

記數符號



Pound Sign

英鎊符號

Sharp

樂譜升記號

Octothorpe

井字遊戲

Comments

```
1 # Single line comments start with a '#'
2 """
3 Multiline strings can be written
4 using three "s, and are often used
5 as comments
6 """
7 # assign the variable s to the object 1
8 s = 1
9 # print out the variable s
10 print(s)
11 s = 1 + 2 # assign s to the result of 1 + 2
12 s = 1 + 2 * 3 - 4 / 5 ### assign s to an expression
13 print(s)
```

Why Comments?

- Describe what is going to happen in a sequence of code
- Document who wrote the code or other ancillary information
- Turn off a line of code - perhaps temporarily

An Example of Comments

```
### The author of the following code is Mr. John Doe ###  
# Get the name of the file and open it  
name = input('Enter file:')  
handle = open(name, 'r')  
text = handle.read()  
words = text.split()  
  
# Count word frequency  
counts = dict()  
for word in words:  
    counts[word] = counts.get(word,0) + 1  
  
# Find the most common word  
bigcount = None  
bigword = None  
for word,count in counts.items():  
    if bigcount is None or count > bigcount:  
        bigword = word  
        bigcount = count  
  
# All done  
print(bigword, bigcount)
```

Variable and Operator

```
a = 95  
a - 3  
print(a)  
a = a - 3  
print(a)
```



```
temp = a - 3  
a = temp
```

```
a = 95  
a -= 3    # a = a - 3  
a += 8    # a = a + 8  
a *= 2    # a = a * 2  
a /= 4    # a = a / 4  
a //= 7   # a = a // 7  
print(a)
```

Operator Precedence Rules

(優先順序)

- **Highest** precedence to **lowest** precedence rule:

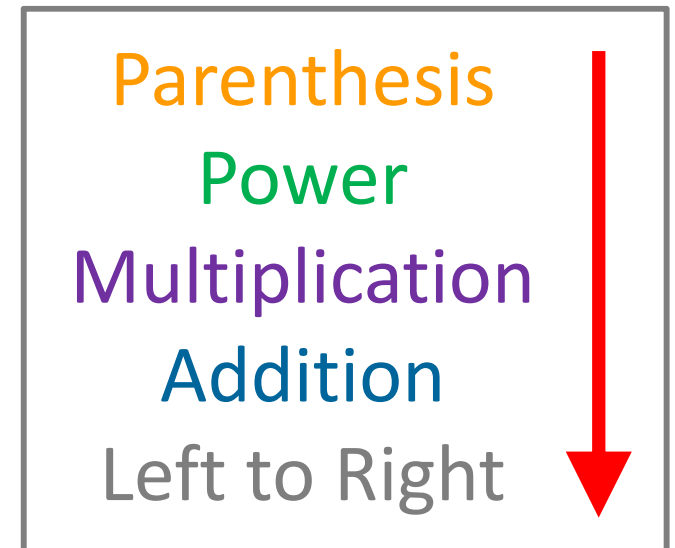
Parenthesis are always respected

> **Exponentiation** (raise to a **power**)

> **Multiplication, Division, and Remainder**

> **Addition and Subtraction**

> **Left to right**



Operator Precedence

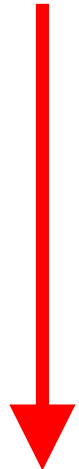
```
x = 1 + 2 ** 3 / 4 * 5  
print(x)
```

```
x = (1 + 2) ** 3 / (4 * 5)  
print(x)
```

```
(1 + 2) ** 3 / (4 * 5)  
3 ** 3 / 20  
27 / 20  
1.35
```

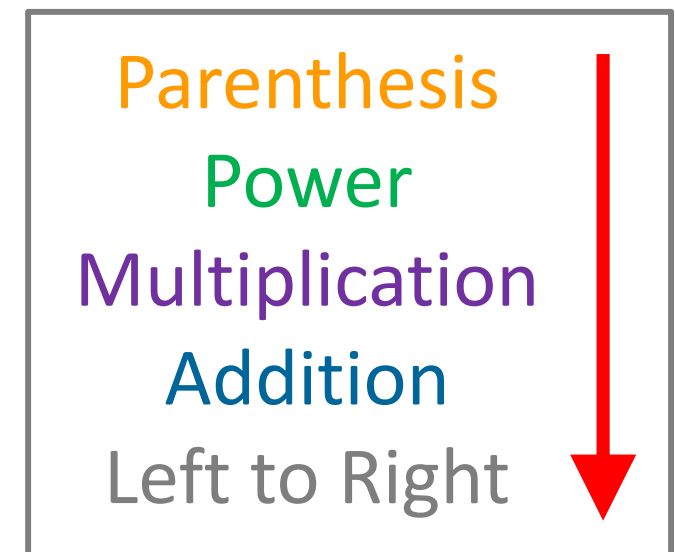
```
1 + 2 ** 3 / 4 * 5  
1 + 8 / 4 * 5  
1 + 2 * 5  
1 + 10  
11
```

Parenthesis
Power
Multiplication
Addition
Left to Right



Principle of Operator Precedence

- Remember the rules top to bottom
- When writing code - **use parenthesis**
- When writing code - keep mathematical expressions simple enough that they are easy to understand
- **Break long series** of mathematical operations up to make them more clear



Break Long Series of Math Operations

```
my_money = 2500 * 20 - 20000 * 2 + 1000 * 3  
# this expression could be confusing
```

```
salary = 2500 * 20
```

```
iphone = 20000 * 2
```

```
part_time = 1000 * 3
```

```
my_money = salary - iphone + part_time
```

```
# this would be better
```

Type Conversion

- In Python variables, literals and constants have a “**type**”
- Python knows the **difference** between an integer number and a string
- For example, “**+**” means “**addition**” if something is a **number** and “**concatenate**” if something is a **string**

```
value = 1 + 5.0
print(value)           # 6.0
another_value = "hello " + "there"
print(another_value)   # hello there
```

Type Conversion

- Python knows what “**type**” everything is
- Some operations are prohibited
- Values with different types cannot be computed
- We can ask Python what type something is by using the **type()** function

```
s = "hello " + "there"
```

```
s = s + 1
```

```
print(s)
```

```
TypeError: Can't convert 'int' object to str implicitly
```

Type Conversion

```
# convert to integer
```

```
int(98.6)      # 98
```

```
type(10e4)     # float
```

```
int(10e4)      # 10000
```

```
type(True)     # boolean
```

```
type(False)    # boolean
```

```
int(True)      # 1
```

```
int(False)     # 0
```

```
int("99")      # 99
```

```
int("-20")     # -20
```

```
int("hello")   ValueError: invalid literal for int() with base 10: 'hello'
```

```
>>> a = 7
>>> b = a
>>> type(a)
<class 'int'>
>>> type(b)
<class 'int'>
>>> type(58)
<class 'int'>
>>> type(99.9)
<class 'float'>
>>> type('abc')
<class 'str'>
```

Get an **error** if the string does not contain numeric characters

Type Conversion

- When you put an integer and floating point in an expression, the integer is **implicitly** converted to a float
- Control types using functions **int()** and **float()**

```
# convert to float
```

```
float(99)          # 99.0
```

```
float("-12.34")    # -12.34
```

```
float(True)        # 1.0
```

```
float(False)       # 0.0
```

```
int("99.5")
```

```
int(float("-99.5"))
```

```
1 + 2 * int(4.5) / 4 - float(int("5"))
```

The print() Function

- To print anything out in the console, use `print()`
 - The parameters of `print()` can be variables, objects, expressions, and statements
 - The parameters of `print()` are separated by **comma**: `,`
 - The printed results are separated by **whitespaces**

```
a = 123
print(a)                # 123
s = "good"
print(a, s)             # 123 good
print(a, s, "morning")  # 123 good morning
print(a + 144 ** 0.5)    # 135.0
b = 77
print(a, "+", b, "=", a + b) # 123 + 77 = 200
```

The print() Function

- By default, the printed values are separated by **whitespaces**
- We can **change the default separation** to an arbitrary string, if we assign this string to the keyword parameter **sep** of the print() function

```
a = 12
b = 34
print(a, b, a + b)           # 12 34 46
print(a, b, a + b, sep=",")  # 12,34,46
print(a, b, a + b, sep=" XDD ") # 12 XDD 34 XDD 46
```

Example

$$ax^2 + bx + c = 0$$

$$x = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

$$x = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

Please write a program (b24ac.py) to **calculate** and **print** the values of x, given **a=5**, **b=13**, **c=-2**

```
◀ ▶ b24ac.py × dist.py × bmi.py
1  # assign values to variables a, b, and c
2  a = 5
3  b = 13
4  c = -2
5
6  # compute each part of the formula
7  b24ac = b**2 - 4*a*c
8  numerator1 = -b + b24ac ** 0.5
9  numerator2 = -b - b24ac ** 0.5
10 denominator = 2 * a
11
12 # obtain the two solutions
13 x1 = numerator1 / denominator
14 x2 = numerator2 / denominator
15
16 # print out the two solutions
17 print(x1)
18 print(x2)
```

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

In-class Exercise

The length of a circle: $2\pi r$

The area of a circle: πr^2

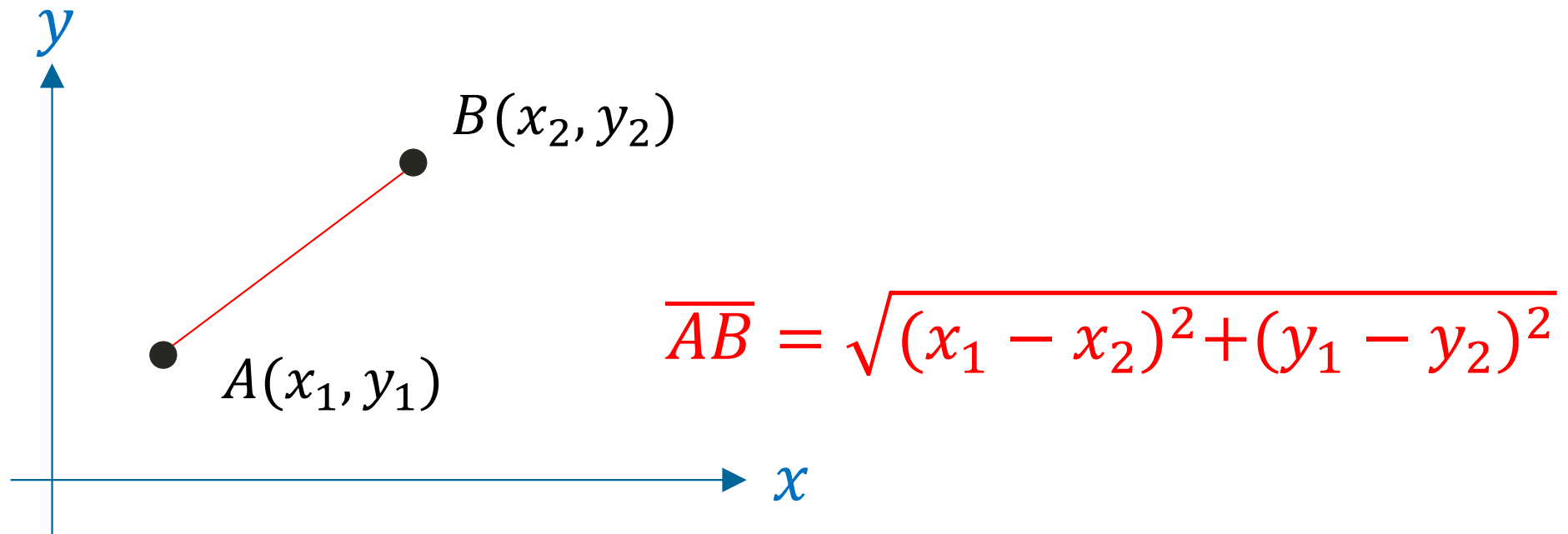
Please write a program (circle.py) to **calculate** and **print** the length and the area of a circle with **radius 5**

Note: π is 3.14

Sample Code for the Exercise

```
1  # assign the floating number 3.14 to the variable pi
2  pi = 3.14
3
4  # assign the variable radius with the integer 5
5  radius = 5
6
7  # compute the length and the area of the circle
8  length = 2 * pi * radius
9  area = pi * (radius ** 2)
10
11 # print out the results
12 print("length =", length)
13 print("area =", area)
```

In-class Exercise



Please write a program (dist.py) to **calculate** and **print** the value of $\overline{AB}$,
given: $(x_1, y_1)=(-3, 4)$ and $(x_2, y_2)=(11, -8)$

Sample Code for the Exercise

```
1  # assign the values of (x1, y1) and (x2, y2)
2  x1, y1 = -3, 4
3  x2, y2 = 11, -8
4
5  # compute the distance between (x1, y1) and (x2, y2)
6  dist_x = x1 - x2
7  dist_y = y1 - y2
8  dist_AB = (dist_x ** 2 + dist_y ** 2) ** 0.5
9
10 # print out the distance
11 print(dist_AB)
```

User Input

- We can instruct Python to pause and read data from the user using the `input()` function
- The `input()` function always returns a **string**
(Please Keep in Mind!!!)

```
name = input("Who are you? ")  
print("Welcome", name)
```

```
c:\Python35-32\test>python test.py  
Who are you? Mary  
Welcome Mary
```

User Input

只要程式裡面有用到`input()`，只能在命令提示字元裡面執行，方能有效地讀取使用者的輸入，在sublime裡面按Ctrl+B或使用Build是無效的。

只要程式裡面有用到`input()`，只能在命令提示字元裡面執行，方能有效地讀取使用者的輸入，在sublime裡面按Ctrl+B或使用Build是無效的。

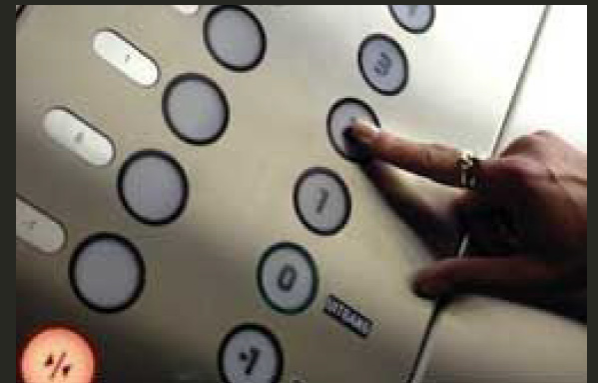
只要程式裡面有用到`input()`，只能在命令提示字元裡面執行，方能有效地讀取使用者的輸入，在sublime裡面按Ctrl+B或使用Build是無效的。

Converting User Input

- If we want to **read a number** from the user, we must **convert it from a string to a number** using a type conversion function, such as `int()` and `float()`

```
inp = input('US floor: ')\nusf = int(inp) - 1      # usf = int(input('US floor: ')) - 1\nprint('Europe floor', usf)
```

```
c:\Python35-32\test>python test.py\nUS floor: 2\nEurope floor 1
```



Summary

- Operator and Expression
- Variables
- Type
- Reserved Words
- Precedence
- Type Conversion
- Comments (#)
- User Input

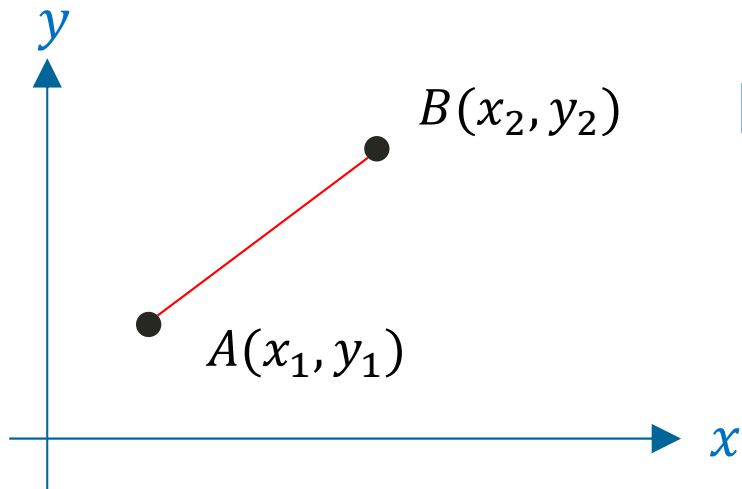
Example: Calculate Your Salary

- Write a program (pay.py) to prompt the user for hours and rate per hour to compute gross pay

```
c:\Python35-32\test>python pay.py
Enter your working hours: 200
Enter the earning rate: 150
You will get 30000.0
```

```
1  # let the user inputs the hour and the rate
2  input_hour = input("Enter your working hours: ")
3  input_rate = input("Enter the earning rate: ")
4
5  # convert the input strings of hour and rate to the floating numbers
6  hour = float(input_hour)
7  rate = float(input_rate)
8
9  # calculate the pay by multiplying the hour and the rate
10 pay = hour * rate
11
12 # print out what you will be paid
13 print("You will get", pay)
```

In-class Exercise: Compute Distance



Let the user input (x_1, y_1) and (x_2, y_2)

$$\overline{AB} = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Write a program (dist_input.py) to **calculate** and **print** the value of $\overline{AB}$, given user-input **floating numbers**

```
c:\Python35-32\workspace>python dist_input.py
Please enter x1: -1.5
Please enter y1: 3.4
Please enter x2: 0.8
Please enter y2: 5.1
The distance between ( -1.5 , 3.4 ) and ( 0.8 , 5.1 ) is: 2.860069929215018
```

In-class Exercise: Compute Distance

```
1 # let the user inputs (x1, y1) and (x2, y2)
2 input_x1 = input("Please enter x1: ")
3 input_y1 = input("Please enter y1: ")
4 input_x2 = input("Please enter x2: ")
5 input_y2 = input("Please enter y2: ")
6
7 # convert the input strings to floating numbers
8 x1 = float(input_x1)
9 y1 = float(input_y1)
10 x2 = float(input_x2)
11 y2 = float(input_y2)
12
13 # compute the distance between (x1, y1) and (x2, y2)
14 dist_x = x1 - x2
15 dist_y = y1 - y2
16 dist_AB = (dist_x ** 2 + dist_y ** 2) ** 0.5
17
18 # print out "The distance between (x1, y1) and (x2, y2) is: dist_AB"
19 print("The distance between (", x1, ",", y1, ") and (", x2, ",", y2, ") is:", dist_AB)
```

In-class Exercise: Body Mass Index (BMI)

- Write a program to prompt the user for height (m) and weight (kg) to compute his/her BMI value

$$\text{BMI} = \frac{\text{weight (kg)}}{\text{height}^2 \text{ (m}^2\text{)}}$$

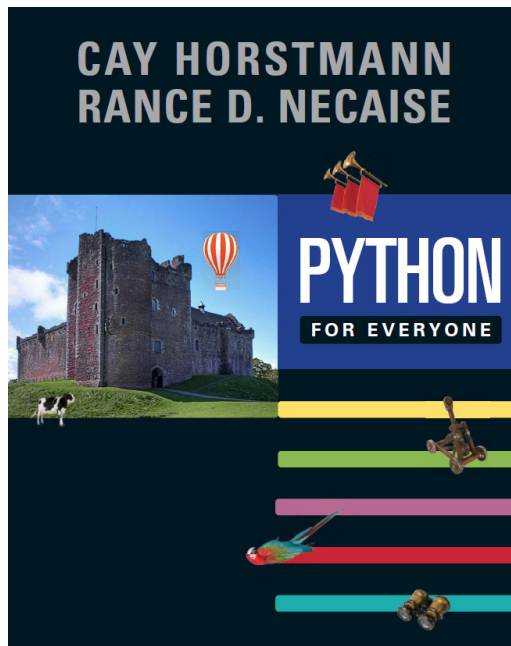
```
c:\Python35-32\test>python bmi.py
Please Input Height (m): 1.72
Please Input Weight (kg): 69.5
Your BMI value is: 23.49242833964305
```

成人肥胖定義	身體質量指數(BMI) (kg/m ²)	腰圍 (cm)
體重過輕	BMI<18.5	
健康體位	18.5 ≤ BMI<24	
體位異常	過重：24 ≤ BMI<27 輕度肥胖：27 ≤ BMI<30 中度肥胖：30 ≤ BMI<35 重度肥胖：BMI ≥ 35	男性：≥ 90公分 女性：≥ 80公分

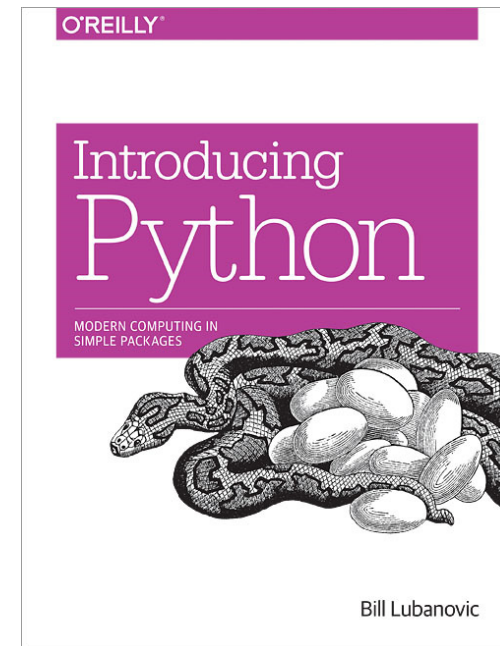
Body Mass Index (BMI)

```
1  # let the user inputs his/her height (m) and weight (kg)
2  input_height = input("Please Input Height (m): ")
3  input_weight = input("Please Input Weight (kg): ")
4
5  # convert the input strings of height and weight to floating numbers
6  height = float(input_height)
7  weight = float(input_weight)
8
9  # calculate the BMI value
10 bmi = weight / (height ** 2)
11
12 # print the result
13 print("Your BMI value is:", bmi)
```

Suggested Reading



P.1 – P.48



P.1 – P.27